

When do you actually use `<=>`?

Contents

1. [Motivation](#)
 - 1.1. [Doesn't quite work in generic code](#)
 - 1.2. [Doesn't quite work in non-generic code](#)
 - 1.3. [... Why not?](#)
 - 1.4. [When do you actually write `<=>`?](#)
 - 1.5. [Reminiscences of `invoke\(\)`](#)
2. [Proposal](#)
 - 2.1. [Examples](#)
3. [Counter-arguments](#)
 - 3.1. [The initial premise is false: `optional<T>` shouldn't always have `<=>`](#)
 - 3.2. [Unintentional comparison category strengthening](#)
4. [References](#)

§ 1. Motivation

[P0515](#) introduced `operator<=>` as a way of generating all six comparison operators from a single function. But the spaceship operator has some usability issues that still need to be addressed.

§ 1.1. Doesn't quite work in generic code

Following from first principles, it would appear that the way you would implement `<=>` for a type like `optional<T>` would look like (writing as a non-member function for clarity):

```
template <typename T>
auto operator<=>(optional<T> const& lhs, optional<T> const& rhs)
    -> decltype(*lhs <=> *rhs)
{
    if (lhs.has_value() && rhs.has_value()) {
        return *lhs <=> *rhs;
    } else {
        return lhs.has_value() <=> rhs.has_value();
    }
}
```

This is a clean and elegant way of implementing this functionality.

But it is wrong.

§ 1.2. Doesn't quite work in non-generic code

One of the big selling features of `<=>` was the ability to simply default it for types that just want to do normal member-by-member lexicographical comparison.

So it seems like this should work:

```
// some perfectly functional C++17 type that implements a total order
struct Ordered {
```

```

bool operator==(Ordered const&) const { ... }
bool operator!=(Ordered const&) const { ... }
bool operator<(Ordered const&) const { ... }
bool operator<=(Ordered const&) const { ... }
bool operator>(Ordered const&) const { ... }
bool operator>=(Ordered const&) const { ... }
};

struct Y {
    int i;
    char c;
    Ordered o;

    auto operator<=>(Y const&) const = default;
};

```

But this doesn't even compile.

1.3. ... Why not?

The problem here is, not all types implement `<=>`. Indeed, at this moment, only the fundamental types do. So writing any sort of code that relies on the existence of `<=>` is highly limited in its functionality.

The provided implementation of `<=>` for `optional` relies on the existence of `<=>` for `T`. As a result, while it works great for `optional<int>`, it would not be a viable candidate for `optional<Ordered>`. Likewise, in order to default the implementation of `<=>` for `Y`, we need each to perform `xi <=> yi` for each member (see [\[class.spaceship\]](#)), but as above, `Ordered` does not implement `<=>`, so this is ill-formed.

What we have to do instead is to use a library function which was also introduced in P0515 but adopted by way of [P0768R1](#): `std::compare_3way()`. What this function does is add more fall-back implementations, in a way best illustrated by this skeleton:

```

template<class T, class U>
auto compare_3way(const T& a, const U& b) {
    if constexpr (/* can invoke a <=> b */)
        return a <=> b;
    else if constexpr (/* can invoke a<b and a==b */)
        return a==b ? strong_ordering::equal : a<b ? strong_ordering::less : strong_ordering::greater;
    else if constexpr (/* can invoke a==b */)
        return a == b ? strong_equality::equal : strong_equality::unequal;
    else
        /* ill-formed, defined as deleted */
}

```

That is, this is an extra library function that lets us use three-way comparisons with types that haven't yet opted into this new language feature. Since the typical case is that types won't have `<=>` implemented, we basically want to make sure that we use `compare_3way()` for the functionality that we need. In other words, the way we want to implement `<=>` for `optional<T>` and for `Y` is:

```

template <typename T>
auto operator<=>(optional<T> const& lhs, optional<T> const& rhs)
    -> decltype(compare_3way(*lhs, *rhs))
{
    if (lhs.has_value() && rhs.has_value()) {
        return compare_3way(*lhs, *rhs);
    } else {
        return compare_3way(lhs.has_value(), rhs.has_value());
    }
}

struct Y {
    int i;
    char c;
    Ordered o;

    strong_ordering operator<=>(Y const& rhs) const {

```

```

18     if (auto cmp = compare_3way(i, rhs.i); cmp != 0) return cmp;
19     if (auto cmp = compare_3way(c, rhs.c); cmp != 0) return cmp;
    return compare_3way(o, rhs.o);
}
};

```

Now, there are places in the above code where I *could* have used `<=>`. I know for a fact that it works for `bool`, `int`, and `char` - so the highlighted lines could have used `<=>` instead of `compare_3way`. But that seems to add so much more cognitive load on the programmer - you have to keep track of what happens to use `<=>` and what doesn't. Just writing `compare_3way` will call `<=>` for you anyway, so why not just always use it?

This is a nuisance for the `optional` implementation, but is arguably worse than that for types like `Y` above - which merely want to opt-in to default semantics. Not only do I have to manually list the implementation, but note that I also have to manually specify the return type! I cannot rely on `auto` to just determine the correct comparison category for me!

Which of course begs the question...

§ 1.4. When do you actually write `<=>`?

P0515 states:

`<=>` is for type implementers: User code (including generic code) outside the implementation of an `operator<=>` should almost never invoke an `<=>` directly (as already discovered as a good practice in other languages); for example, code that wants to test `a<b` should just write that, not `a<=>b < 0`.

I absolutely agree with the intent of this statement - user code should use binary operators. But as illustrated above, type implementers can't use `<=>` either! At least, not until every type the implements an ordering does so by way of `<=>`. Which is to say, never.

Effectively, there are exactly two places that can correctly use `<=>`:

1. The compiler, which would transform expressions like `a<b` into `a<=>b < 0`.
2. The implementation of `std::compare_3way()`.

That seems a waste of a perfectly good token that we have specially reserved for this occasion. `<=>` just seems like a much better spelling than `compare_3way`, especially given its ability to be used as an infix operator.

§ 1.5. Reminiscences of `invoke()`

We already have one example where we have a language feature that doesn't quite do everything we need it to do, so we have a library feature that fills in the gaps and thus needs to be used unconditionally: function calls and `std::invoke()`. In generic code today, anything constrained with `std::result_of` (before C++17), `std::invoke_result` or `std::is_invokable` (C++17 or later), or `Invocable` or `RegularInvocable` (C++20 after P0898 needs to use `std::invoke()` to instead of normal function call syntax. Otherwise, types that meet the constraint but aren't usable with normal function call syntax (i.e. pointers to member functions and pointers to member data) will trigger hard errors.

I continue to view this as an unfortunate split, but we get away with this as a language because pointers to members are fairly rare compared to functions and functions objects, so the usual function call syntax just works the vast majority of the time. Additionally, function call syntax is used in user code all the time.

By contrast with `<=>`, the common case is types *not* supporting `<=>` and `<=>` won't be commonly used in user code. So it is both true that the gaps that `compare_3way()` is filling are much more significant than the equivalent gaps that `invoke()` is filling as well as `<=>` being less useful in user code than normal function call syntax.

§ 2. Proposal

This paper proposes to move the entirety of the logic of `compare_3way()` into the specification of `operator<=>()`. In other words, ignoring parameter inversion, `a <=> b` shall mean:

1. Lookup `a.operator<=>(b)` and `operator<=>(a, b)` as usual for binary operators. If a viable candidate is found, we are done.
2. Otherwise, if `a == b` and `a < b` are each well-formed and convertible to `bool`, then the expression has type `strong_ordering` with value `(a == b) ? strong_ordering::equal : ((a < b) ? strong_ordering::less : strong_ordering::greater)`

3. Otherwise, if `a == b` is well-formed and convertible to `bool`, then the expression has type `strong_equality` with value `(a == b) ? strong_equality::equal : strong_equality::nonequal`.
4. Otherwise, the expression is ill-formed

This paper additionally proposes to turn `compare_3way` into a function object class that simply invokes `<=>` on its two arguments, along the lines of `less`.

With this change, we would actually be able to use `<=>` in all the places where we want to use `<=>` today but cannot. We would also be able to default the implementation of `<=>` instead of having to manually implement what should be the defaulted implementation.

§ 2.1. Examples

Today (P0515/P0768/C++2a)	Proposed
<pre>template <typename T> auto operator<=>(optional<T> const& lhs, optional<T> const& rhs) -> decltype(compare_3way(*lhs, *rhs)) { if (lhs.has_value() && rhs.has_value()) { return compare_3way(*lhs, *rhs); } else { return compare_3way(lhs.has_value(), rhs.has_value()); } }</pre>	<pre>template <typename T> auto operator<=>(optional<T> const& lhs, optional<T> const& rhs) -> decltype(*lhs <=> *rhs) { if (lhs.has_value() && rhs.has_value()) { return *lhs <=> *rhs; } else { return lhs.has_value() <=> rhs.has_value(); } }</pre>
<pre>struct Y { int i; char c; Ordered o; strong_ordering operator<=>(Y const& rhs) const { if (auto cmp = compare_3way(i, rhs.i); cmp != 0) return cmp; if (auto cmp = compare_3way(c, rhs.c); cmp != 0) return cmp; return compare_3way(o, rhs.o); } };</pre>	<pre>struct Y { int i; char c; Ordered o; auto operator<=>(Y const&) const = default; };</pre>

§ 3. Counter-arguments

There are two counter-arguments I'm aware of to this proposal. What follows is an accounting of those arguments and my responses to them.

§ 3.1. The initial premise is false: `optional<T>` shouldn't always have `<=>`

`optional<T>` only has `<` if `T` has `<`, so `optional<T>` should only have `<=>` if `T` has `<=>`. In other words, the initial provided implementation is the correct implementation and the suggested one using `compare_3way()` is incorrect.

There isn't anything particularly specific to `optional` in this argument, so we can extend this to: Any compound type should have `<=>` only if all of its constituents have `<=>`.

This argument sounds seductive, and offers consistency with the way we write other operators. But it has some serious problems.

To start with, the implication here is that `optional<T>` (and by extension every compound type) would need now to conditionally implement *seven* operators (all six pre-existing comparisons, plus `<=>`) instead of the advertised *one* operator. As a result, we lose a big advantage from `<=>`: the ability to write less code.

The consequence of writing all seven operators is that it makes `<=>` oddly useless. The only code that should invoke `<=>` is other types' implementations of `<=>`... so in order to invoke `optional<T>`'s `<=>` for a `T` that provides that operator, I would need to have a type that has such a thing as a member. In other words:

```

struct WithSpaceship {
    // ...
    strong_ordering operator<==>(WithSpaceship const& rhs) const {
        // ...
    }
};

struct X {
    optional<WithSpaceship> opt_ws;
    auto operator<==>(X const&) const = default;
};

X x;
x.opt_ws < x.opt_ws; // calls optional's <, not <==>
x < x;               // calls optional's <==> via X's <==>

```

This is, in of itself, odd. Not only are we not saving code, but the extra operator that we're writing will rarely be used.

But the bigger consequence of this is that we lose the ability to improve performance with three-way comparisons. Typically, `<==>` can be better than consecutive calls to `<`. Consider `pair<T, T>`. If we're only providing `<==>` for such a type if `T` has `<==>`, that means we must still be providing `<` if `T` has `<`. If `T` has `<==>`, `T` has `<`, which means that our `pair<T, T>` provides everything.

That looks something like this:

```

// < as it exists today, approximately
template <typename T, typename U>
bool operator<(pair<T,U> const& x, pair<T,U> const& y)
{
    return x.first < y.first ||
        (!(y.first < x.first) && x.second < y.second);
}

// <==> as it would exist based on this reasoning
template <typename T, typename U>
auto operator<==>(pair<T,U> const& x, pair<T,U> const& y)
    -> common_comparison_category_t<...>
{
    if (auto cmp = x.first <==> y.first; cmp != 0) return cmp;
    return x.second <==> y.second;
}

pair<WithSpaceship, WithSpaceship> p = ...;
p < p;    // calls pair::operator<
p <==> p; // calls pair::operator<==>

```

`WithSpaceship` has an efficient `<==>`, but `pair`'s `operator<` can't use it! Which means that `p < p` has to call `<` three times, which in turn calls `WithSpaceship::operator<==>` three times. But `p <==> p` only has to call `<==>` twice.

This argument could be extended out to `vector<T>`'s `operator<` which now potentially does up to $2N-1$ calls to `T`'s `<` instead of up to N calls to `T`'s `<==>`. That could be a serious pessimization - one that would encourage people to actually write `<==>` in code! After all, if `(a <==> b) < 0` could potentially be faster than `a < b`, why would I write the latter? That would be a serious design error.

In short, only conditionally providing `<==>` for compound types not only defeats the goal of writing less code, but also defeats the goal of writing more performant code. I think the only conclusion is that compound types need to provide `<==>` whenever their underlying types are comparable at all and be an unconditional substitute for the other relational operators.

§ 3.2. Unintentional comparison category strengthening

When a class author implements `<==>` for their type, they have to decide what comparison category to use as the return type. Other code could use that choice to make important decisions. But if we had `<==>` fall-back to `compare_3way()`, we effectively are guessing what the intended comparison category was. `decltype(x <==> y)` might be a deliberate choice of the class author - or it might be compiler inference, and we can't tell.

Moreover, we might get this wrong in a very confusing way that inadvertently strengthens the comparison category. For example, the following code is well-formed:

```
bool foo(error_code const& a, error_condition const& b) {  
    return a == b;  
}
```

because there exists an equality comparison between these [two types](#). These two types mean rather different things and obviously are not substitutable, but the proposed change would nevertheless give `decltype(a <=> b)` the type `strong_equality` and that is a very misleading and strongly undesired result.

I have two responses to this argument.

First, practically speaking there is no difference between using `a <=> b` in this context and getting `strong_equality` and using `compare_3way(a, b)` in this context and getting `strong_equality`. The end result is the same - equally misleading, and the added weight on the meaning of `<=>` here is a distinction without a difference. In today's world, people will write `<=>` when they need a three-way comparison, find that it doesn't work, and then switch to `compare_3way()` - because `compare_3way()` solves a problem.

Moreover, even if we maintained that `a <=> b` is ill-formed (as it is today), that wouldn't change the fact that the provided valid implementation of `optional` using `compare_3way()` would give already us `optional(a) <=> optional(b)` as a valid operation whose type is `strong_equality`. As argued in the previous section, `optional` needs to implement `<=>`, and it needs to implement it in this way, so this comparison category strengthening already exists as an issue. I'm unconvinced that *specifically* `<=>` yielding a misleading response *specifically* on the underlying types is a problem.

Second, the fact that `compare_3way(a, b)` today and `a <=> b` with this proposal yields `strong_equality` is a problem - but it's not `<=>`'s problem, it's `error_code`'s problem. The decision to use `==` in this context is arguably a bad design decision. With the direction the standard library is going and the new concepts in Ranges in [P0898](#) (both terminology concepts and language concepts), we are adding semantic requirements on top of syntactic ones - and the lack of substitutability between `error_code` and `error_condition` means that we're meeting the syntactic but not the semantic requirements of `EqualityComparableWith`. This is bad.

Jonathan Müller recently wrote an in-depth series of blog posts on the [mathematics behind comparisons](#) which makes the compelling argument that the existence of `==` should be `strong_equality` only, and the existence of `==` and `<` should be `strong_ordering` only. Any other required comparison category should be a named function instead. This argument is very much in line with the principles behind Ranges.

The conclusion of this is that yes, there is a problem. But the problem lies with `error_code` and `error_condition` for violating expectations with the decision to improperly use `==` when it doesn't mean equality and substitutability. Having `<=>` fall back to guessing at the comparison category and yielding `strong_ordering` or `strong_equality` works for types that follow good design guidance in their choice of using operators.

Note that this problem could easily be fixed by replacing the currently existing `==` and `!=` for these types with a `<=>` returning `weak_equality`. This is still a questionable choice, but at least you would observe the correct comparison category without otherwise breaking user code. Ideally, `==` and `!=` get replaced with a named function that itself returns `weak_equality`.

This leaves the question of whether or not `<=>` was explicitly provided or inferred. In practice, I think this is about as relevant as whether or not the copy constructor was explicit or compiler generated (or more relevantly, as whether or not `<` was generated from `<=>` or from `<`). As long as it has sane semantics. However, if people feel strongly about wanting this particular piece of information, if we're already adding language magic to have `<=>` perform multiple possible operations, it is surely possible to add language magic to directly retrieve the type of the actual binary `<=>` operation if and only if it exists.

§ 4. References

- [\[P0515R3\]](#) "Consistent comparison" by Herb Sutter, Jens Maurer, Walter E. Brown, 2017-11-10
- [\[P0768R1\]](#) "Library Support for the Spaceship (Comparison) Operator" by Walter E. Brown, 2017-11-10
- [\[P0898R3\]](#) "Standard Library Concepts" by Casey Carter, Eric Niebler, 2018-06-08
- [\[alg.3way\]](#) Current Working Draft
- [\[class.spaceship\]](#) Current Working Draft
- [\[muller.compare\]](#) "Jonathan Müller's blog() - Mathematics behind Mathematics behind Comparison #4: Three-Way Comparison" by Jonathan Müller, 2018-09-07

- [\[syserr.compare\]](#) Current Working Draft